

Sinwave – Architecture du jeu

Projet Aegis · B3 Workshop 1 · GZDoom 4.14.2, ZScript · Oct 1, 2026 · @moi

Le jeu en bref

Sinwave est un survivors-like en vue subjective sur les sept péchés capitaux : le joueur descend des cercles de l'Enfer, survit à des vagues d'ennemis, et chaque choix fait pencher son âme. Il tourne sur GZDoom 4.14.2 et il est écrit en ZScript. Tout le contenu du jeu est décrit dans des fichiers texte, hors du code.

Moment	Ce qui se passe
Hors run	Choix de l'arène : le Vestibule (2 à 3 min), le Purgatoire (environ 8 min), les Limbes (environ 6 min) ou une arène personnalisée. Réglage de la descente : difficulté, défi personnalisé, cercle de départ. Boutique des indulgences.
Une run	Des cercles, un par péché. Chaque cercle impose une malédiction et enchaîne des vagues de plus en plus dures. Un boss ferme le dernier cercle : Charon au Vestibule, Lucifer au Purgatoire.
À chaque niveau	Trois choix : une vertu (petit bonus, l'âme se purifie), un péché (gros bonus avec un défaut, l'âme se corrompt) et souvent un neutre (bonus moyen, l'âme ne bouge pas).
L'âme	Une balance de 0 (sainteté) à 30 (damnation), départ à 15. Trois paliers de chaque côté changent la partie. Le verdict final en découle : Absolution, Purgatoire ou Damnation.
À la fin	Les indulgences gagnées sont sauvegardées. Le Jugement de l'âme bouge et ouvre d'autres rayons de la boutique.

Choix du moteur : pourquoi GZDoom

GZDoom fournit déjà un jeu de tir complet mais aucune architecture de jeu : le temps va donc à ce que le sujet note, et tout ce qu'il demande est écrit et visible dans le projet. Le sujet note surtout l'architecture (états, bus, services, données, méta-progression) et

demande une run jouable de 2 à 3 minutes.

	GZDoom 4.14 (ZScript)	Unity (C#)	Godot 4 (GDScript, C#)	Unreal 5 (C++, Blueprints)
Jeu de tir de base : déplacement, armes, monstres et leur IA, collisions, sons, menus, manette	fourni	à écrire, ou paquets de l'Asset Store	à écrire	modèle FPS fourni ; ennemis et vagues à écrire
Graphismes et sons	Freedoom, libre et redistribuable	à trouver ou à produire	à trouver ou à produire	à trouver ou à produire
Langage	ZScript : classes, héritage, méthodes virtuelles et abstraites, typage statique	C#	GDScript (typage facultatif), C#	C++, Blueprints
Architecture fournie par le moteur	presque rien : tout est écrit dans le projet	ScriptableObject, UnityEvent	signaux, autoloads	sous-systèmes, délégués
Poids	mod de 110 Ko, build Windows complet de 31 Mo	éditeur et build bien plus lourds	léger	très lourd
Itération	tout le ZScript compile en quelques secondes	recompilation et rechargement à chaque modification	rapide	compilation C++ longue

Pourquoi GZDoom :

- **Le temps va à ce qui est noté.** Pas de contrôleur de personnage, d'armes ni d'IA de monstres à écrire avant de pouvoir travailler l'architecture.
- **L'architecture ne peut pas être empruntée au moteur.** GZDoom n'a ni bus d'événements, ni machine à états de jeu, ni équivalent des ScriptableObject.

- **ZScript est un vrai langage objet**, vérifié à la compilation et proche du C# : Observer, State, Stratégie et Service Locator s'y écrivent comme en Unity.
- **La séparation logique / affichage est garantie par le compilateur** : le code `ui` peut lire le jeu, pas le modifier.
- **Le genre s'y prête** : vagues de monstres, armes, expérience. Deux mods survivors-like existent déjà (DoomSurvivor, Doom2Survive) ; Sinwave s'en distingue par une descente finie, des malédictions par cercle, un choix moral et une boutique entre les runs.
- **Un build léger et libre** : GZDoom (GPL) et Freedoom (BSD) se redistribuent ; le build se lance par un `.bat`.

Pourquoi pas les autres : Unity et Godot donnent plus de liberté (vraie 3D, éditeur, débogueur), mais il aurait fallu construire tout le jeu de tir avant d'arriver à ce qui est évalué. Unreal est surdimensionné pour un prototype court, et le C++ ralentit chaque essai.

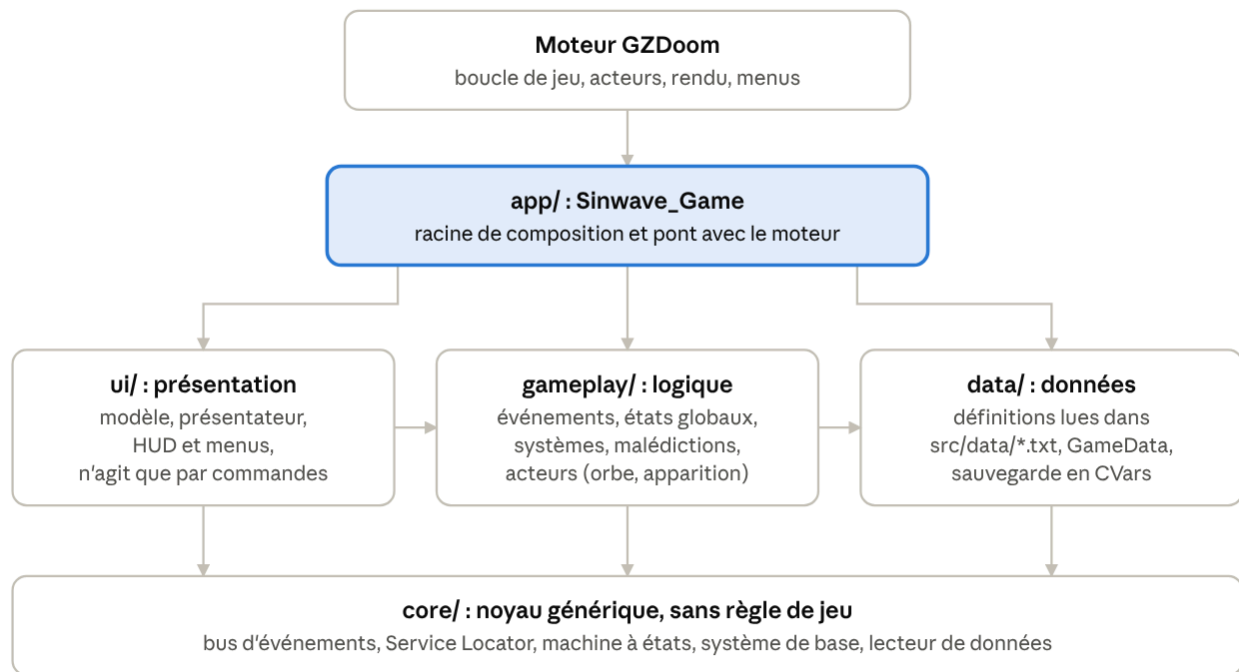
Ce que ce choix coûte :

- ZScript n'a ni génériques, ni délégués, ni lambdas, ni constructeurs avec paramètres : les abonnés du bus héritent d'une classe, et un Service Locator transmis remplace un conteneur d'injection.
- Ni débogueur, ni framework de test : les tests sont écrits dans le projet et ouvrent une fenêtre GZDoom.
- Un moteur de 1993 en 2,5D : cartes faites de secteurs, monstres en sprites, IA sans recherche de chemin. Les arènes doivent rester ouvertes.
- ZScript ne peut pas écrire de fichier : la sauvegarde passe par des CVars, dans le `.ini` du moteur.
- Il faut un IWAD (les données de base) : Freedoom est livré ; le vrai Doom II marche aussi s'il est installé.

Les couches du code

Le code est rangé en cinq couches dont les dépendances descendent toujours : le noyau générique ne connaît rien du jeu, et seule la racine de composition connaît toutes les classes concrètes.

Les dépendances descendent toujours : le noyau ne connaît rien du jeu



Flèche : « dépend de ». En couleur : la racine de composition, seule à connaître toutes les classes.

les couches du code · 5 couches, dépendances vers le bas

Les données ne connaissent pas les systèmes : une malédiction y est citée par le nom de sa classe, vérifié au chargement. Hors du code : `src/data/` (contenu), `src/maps/` (cartes), `tests/`, `tools/`.

Patron de conception	Où	Rôle
Racine de composition	<code>Sinwave_Game</code>	Un seul endroit crée et relie tous les objets
Observer (bus d'événements)	<code>Sinwave_EventBus</code>	Les systèmes communiquent sans se connaître
Service Locator transmis	<code>Sinwave_Services</code>	Accès aux services sans singleton statique
State	<code>Sinwave_StateMachine</code> et ses états	Les états globaux du jeu et leurs transitions
Stratégie	<code>Sinwave_Curse</code> et ses 7 sous-classes	La règle de chaque cercle, choisie par les données
Modèle-Présentateur-Vue	<code>ui/</code>	L'interface lit un modèle et n'agit que par des commandes

Patron de conception	Où	Rôle
Conception orientée données	src/data/	Tout le contenu est décrit hors du code

Architecture du code : les classes

Ces diagrammes montrent les vraies classes du code (src/zscript/) avec leurs principaux membres, comme un diagramme de classes UML. Toutes les classes du projet commencent par Sinwave_ , car ZScript n'a pas d'espaces de noms.

Dans les schémas	Sens
Cadre blanc	La classe, et sous son nom la classe dont elle hérite
Cadre vert	Ses membres : champs (mNom) puis méthodes
Trait plein, triangle vide	« hérite de »
Pointillés, flèche	« utilise » ou « crée »

Le noyau

Le noyau : tout passe par le bus, sans accès global

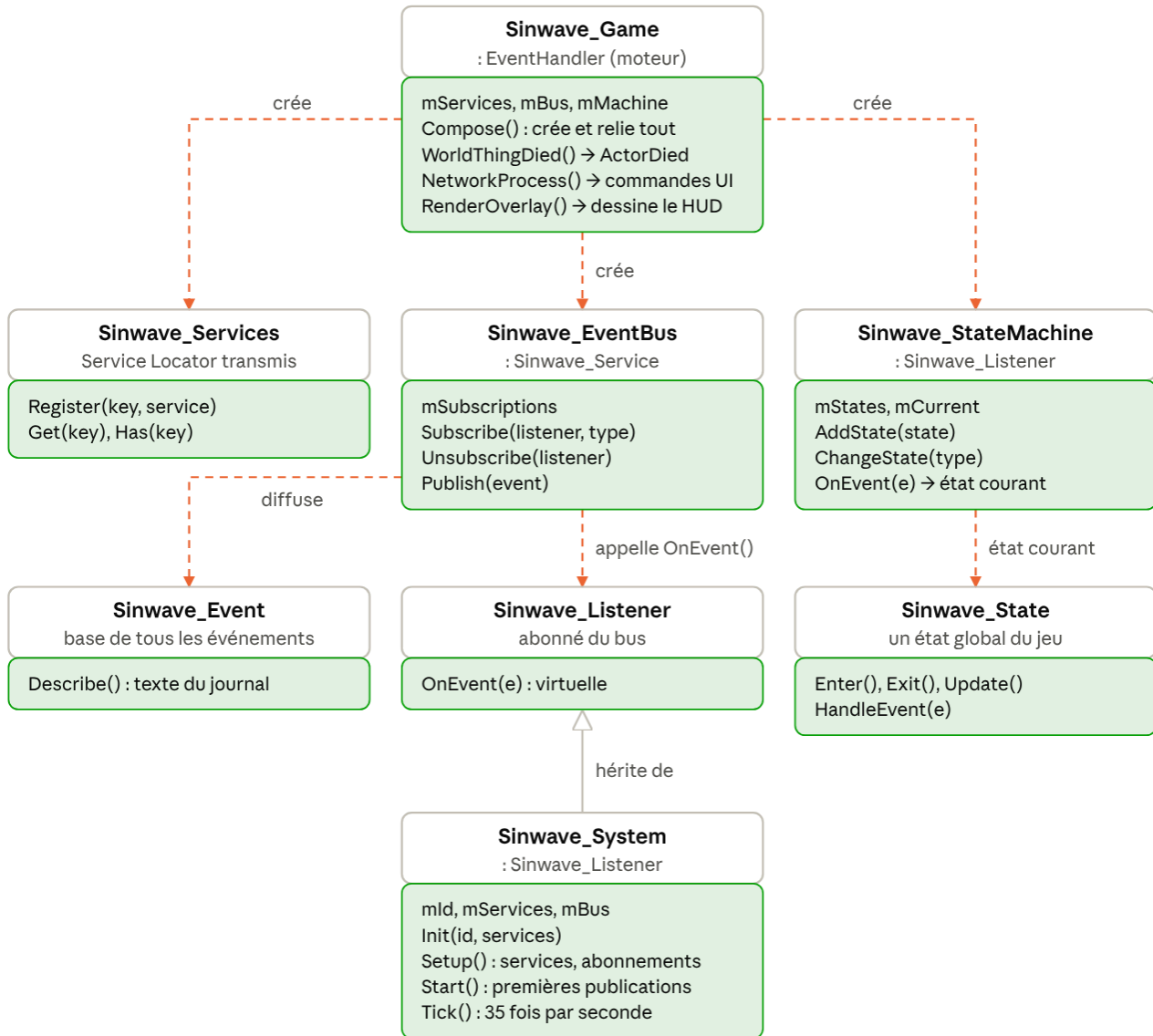
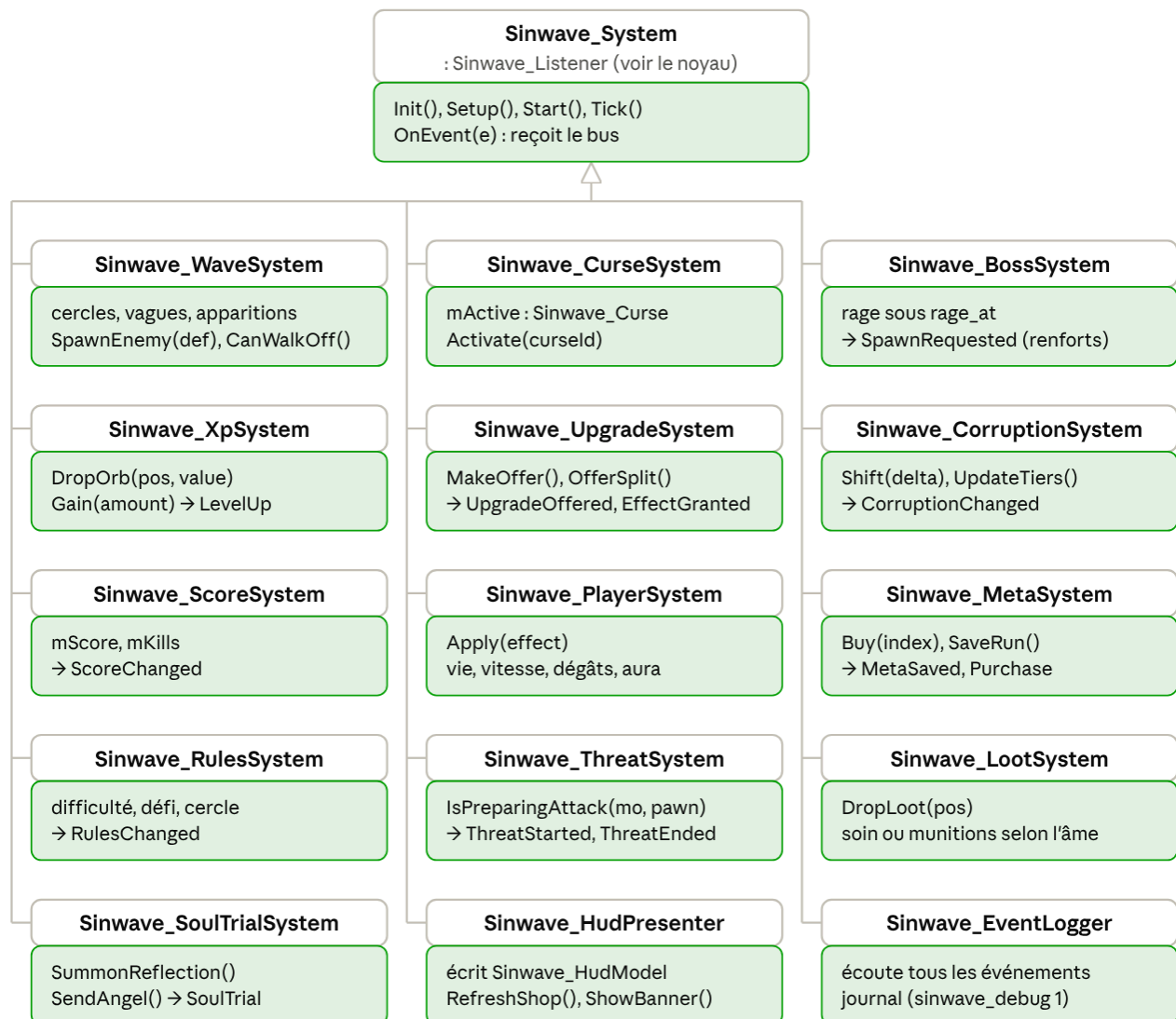


diagramme de classes · le noyau (core/ et app/)

Sinwave_Game est la seule classe connue du moteur : elle crée les services, les états et les systèmes, puis traduit les appels du moteur en événements. Le bus, la machine à états et les systèmes héritent tous de **Sinwave_Listener** : ils reçoivent les événements par la même méthode, **OnEvent()**.

Les systèmes de jeu

15 systèmes, une seule classe de base : aucun ne connaît les autres



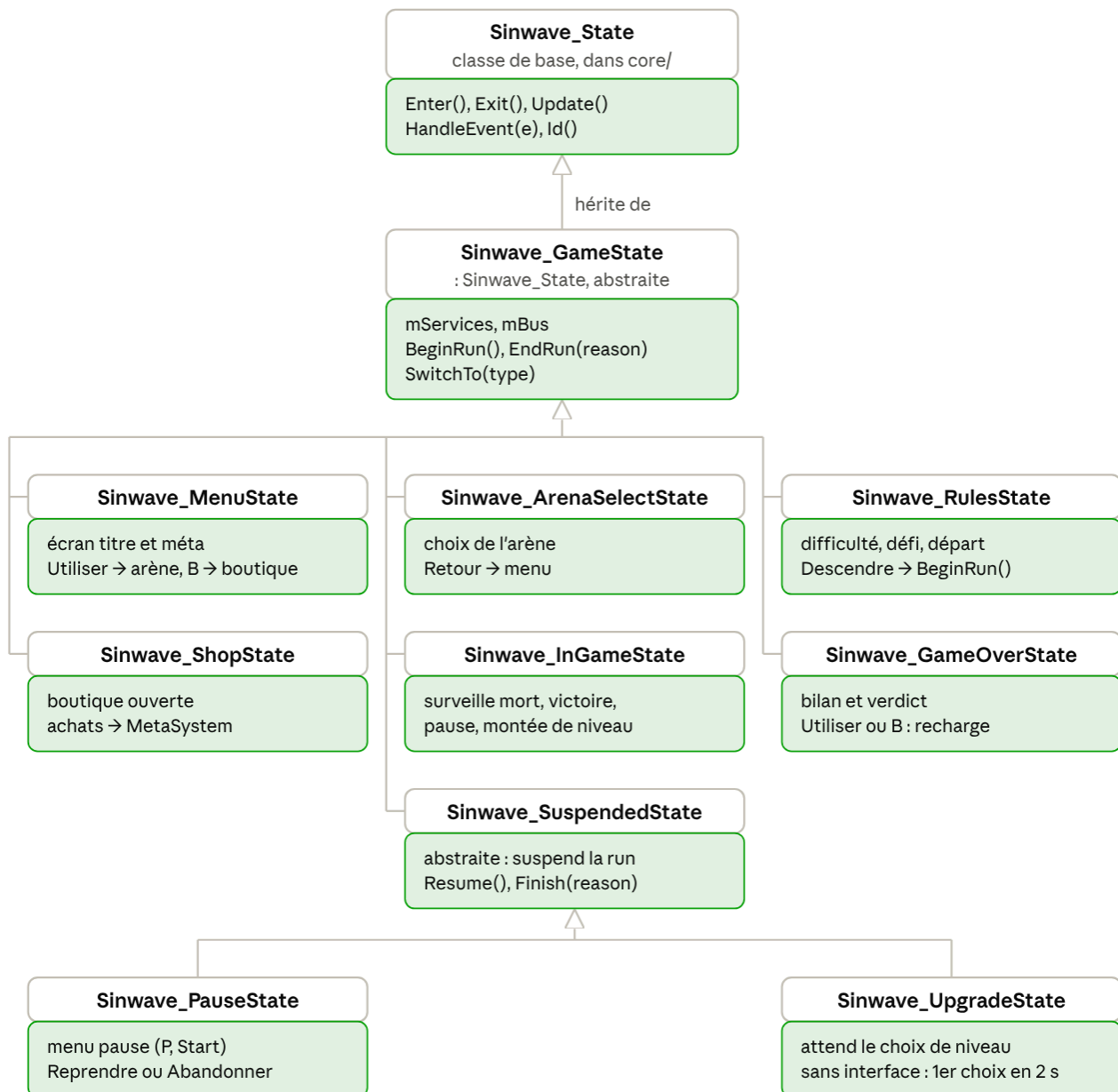
Chacun est créé d'après data/systems.txt : enabled = false le coupe, une autre class = le remplace.

diagramme de classes · les 15 systèmes de jeu

Chaque système ne fait qu'une chose et ne parle qu'au bus : une flèche « → » indique les événements qu'il publie. **Sinwave_HudPresenter** est un système comme les autres, ce qui permet de jouer sans interface dans les tests.

Les états

Les états décident des transitions, les systèmes font le travail



Toutes les transitions passent par Sinwave_StateMachine.ChangeState() : Exit(), StateChanged, puis Enter().

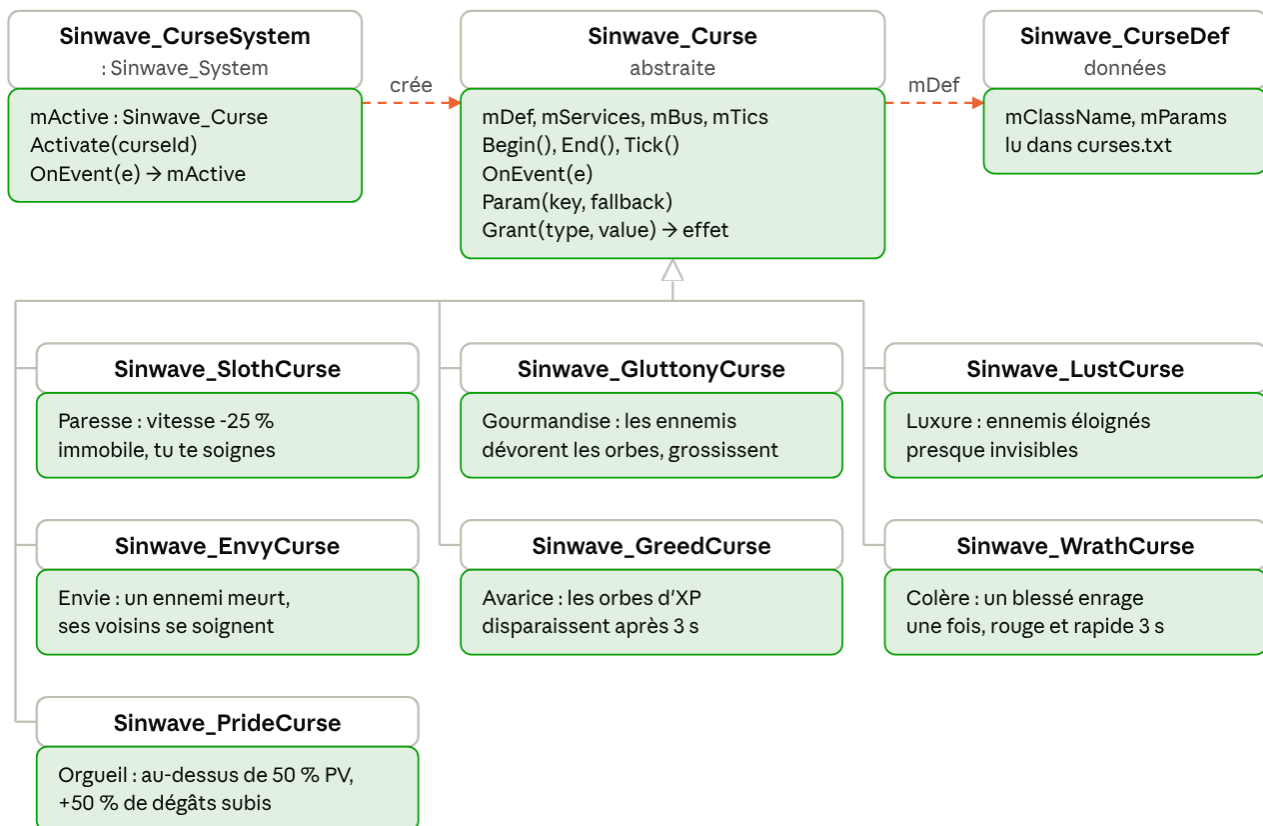
diagramme de classes · les états globaux

Le code commun n'est écrit qu'une fois : le début et la fin de run dans

`Sinwave_GameState`, la suspension et la reprise dans `Sinwave_SuspendedState`, dont héritent Pause et Amélioration.

Les malédictions

Pattern Stratégie : la règle de chaque cercle est interchangeable



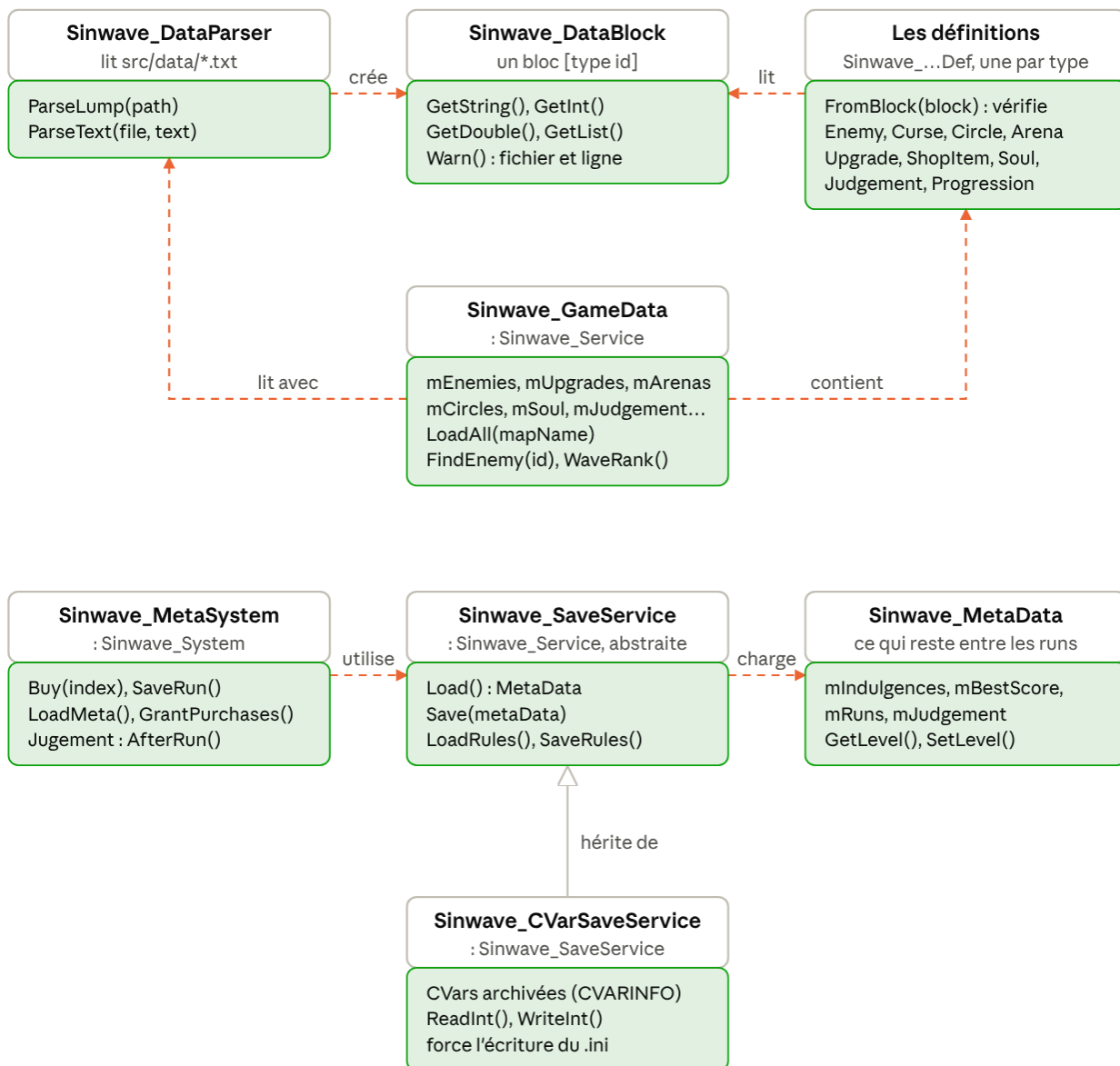
Choisie par son nom dans data/curses.txt : une nouvelle malédiction = une sous-classe + un bloc de données.

diagramme de classes · les 7 malédictions

Sinwave_CurseSystem ne sait pas ce que fait une malédiction : il crée la classe nommée dans les données, l'active au début du cercle et lui transmet le bus jusqu'à la fin. Un effet sur le joueur passe par `Grant()`, qui publie `EffectGranted`, puis son inverse à la fin du cercle.

Données et sauvegarde

Données et sauvegarde : lues et écrites derrière des services



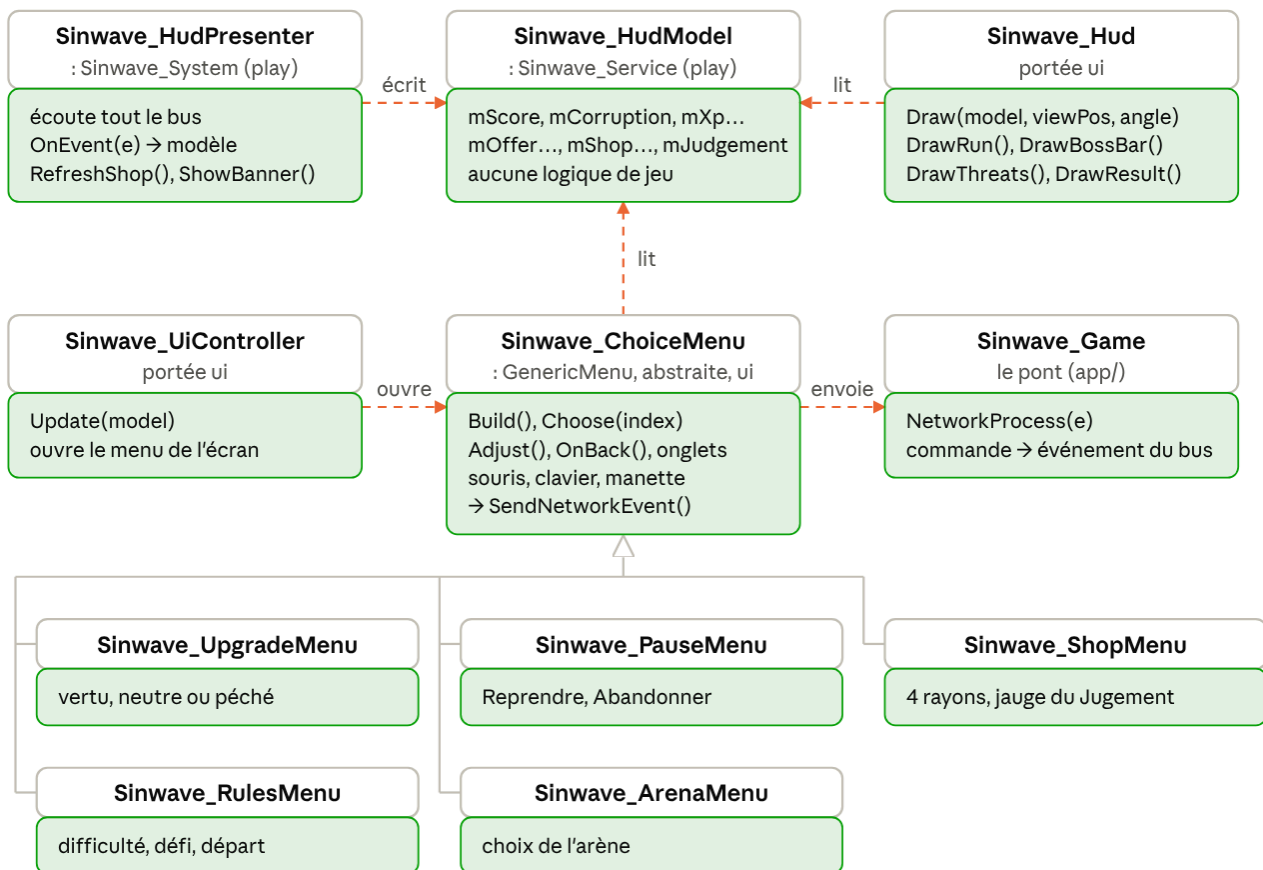
Un autre stockage (fichier, serveur) serait une autre classe qui hérite de Sinwave_SaveService.

diagramme de classes · données et sauvegarde

Les systèmes ne lisent jamais un fichier : ils demandent `Sinwave_GameData` au Service Locator. La méta-progression ne connaît que le contrat abstrait `Sinwave_SaveService`, pas les CVars.

L'interface

L'interface lit un modèle et n'agit que par des commandes



Le code ui peut lire les objets play mais pas les modifier : le compilateur de ZScript le garantit.

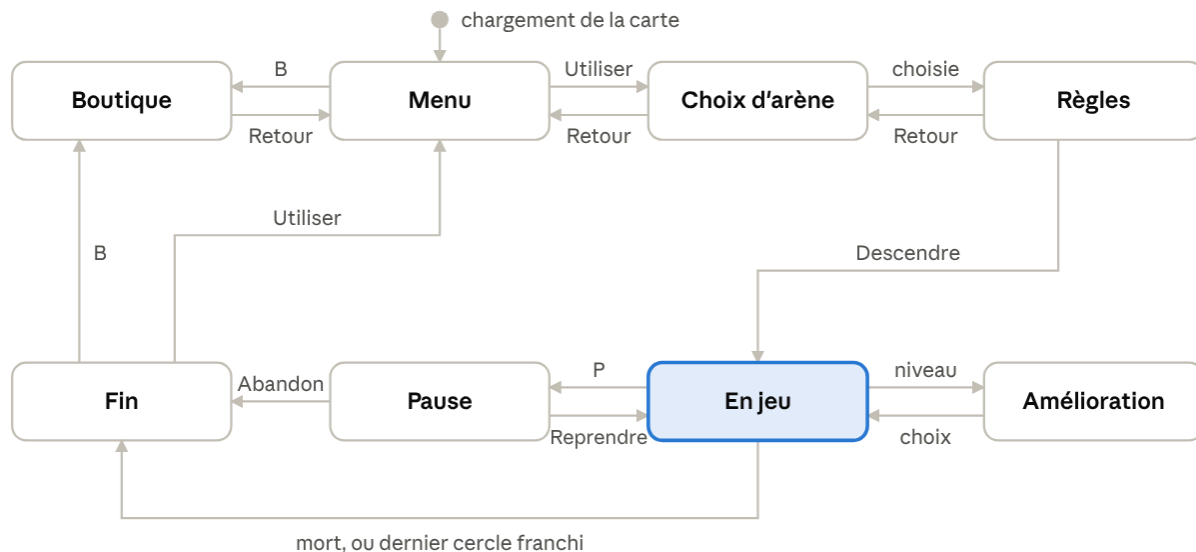
diagramme de classes · l'interface (modèle, présentateur, vues)

C'est un modèle-présentateur-vue : le présentateur écrit le modèle, le HUD et les menus le lisent, et les menus n'agissent que par des commandes que `Sinwave_Game` traduit en événements.

La machine à états du jeu

Huit états globaux décident de ce que le joueur peut faire, et toutes les transitions passent par une seule fonction : `Sinwave_StateMachine.ChangeState()`, qui appelle `Exit()`, publie `StateChangedEvent`, puis appelle `Enter()`.

Huit états globaux : la run tourne autour de « En jeu »



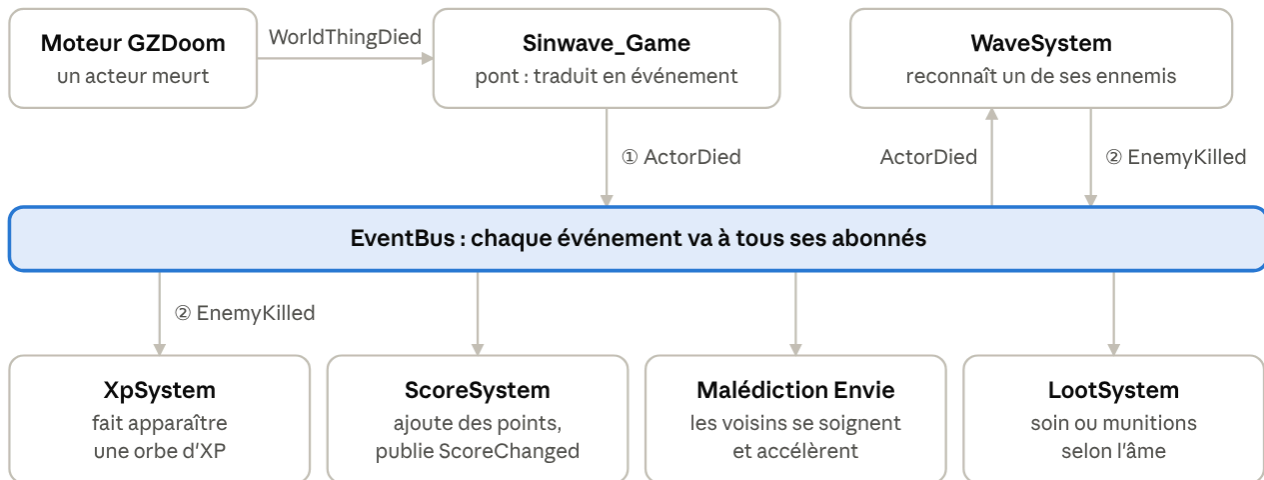
machine à états globale · 8 états

- **Les états décident, les systèmes travaillent.** Quand la run démarre, l'état publie `RunStartedEvent` ; ce sont les systèmes qui réagissent (cercles, joueur, XP, achats appliqués).
- **Pas de code dupliqué :** le début et la fin de run sont écrits une fois dans `Sinwave_GameState` ; Pause et Amélioration héritent de `Sinwave_SuspendedState`, qui suspend et reprend la run.
- **Changer de carte :** une arène sur une autre carte démarre après un voyage. L'état note l'action dans une CVar non sauvegardée (`sinwave_onarrival`), relue à l'arrivée. Depuis Fin, la carte se recharge sur le menu (Utiliser) ou sur la boutique (B).
- La machine est générique (`core/`) : elle ne connaît que `Sinwave_State` et transmet chaque événement du bus à l'état courant.

Le bus d'événements : les systèmes ne se connaissent pas

Les systèmes ne s'appellent jamais entre eux : ils publient des événements sur `Sinwave_EventBus` et s'abonnent à ceux qui les intéressent (pattern Observer). Tous les événements du jeu sont réunis dans un seul fichier, `gameplay/sinwave_events.zs` : le lire suffit pour savoir tout ce qui peut se passer.

Un ennemi meurt : quatre systèmes réagissent sans se connaître



Aucun de ces systèmes ne référence les autres : ils ne connaissent que le bus et les classes d'événements.

La suite montre la même chaîne : quand le joueur ramasse l'orbe, elle publie `XpCollected` ; `XpSystem` publie `LevelUp` ; l'état Amélioration suspend la run et `UpgradeSystem` propose trois choix.

```

mBus.Subscribe(self, 'Sinwave_EnemyKilledEvent'); // s'abonner à un type
mBus.Publish(Sinwave_EnemyKilledEvent.Create(def, pos)); // publier
  
```

- Un abonnement porte sur une classe d'événement et ses sous-classes ; sans type, l'abonné reçoit tout (journal, présentateur de l'interface).
- La diffusion est **synchrone** : `Publish()` appelle les abonnés dans l'ordre d'abonnement, et un abonné peut publier à son tour.
- Un système qui a besoin d'un autre le **demande** par un événement : en rage, le boss publie `SpawnRequested` et c'est `WaveSystem` qui fait apparaître les renforts.
- ZScript n'a ni délégués ni `event` comme C# : les abonnés héritent de `Sinwave_Listener` et redéfinissent `OnEvent()`.

Services transmis, sans singleton

Aucun objet n'est accessible de partout : la racine de composition `Sinwave_Game` crée un Service Locator (`Sinwave_Services`) et le transmet à chaque état, système et malédiction quand elle le crée (`Init(services)`). Le sujet interdit le singleton statique ; ici, un code qui n'a pas reçu le locator ne peut rien retrouver.

Clé	Service	Rôle
EventBus	Sinwave_EventBus	Communication entre systèmes
GameData	Sinwave_GameData	Données du jeu et arène courante, en lecture seule
Save	Sinwave_CVarSaveService , derrière le contrat abstrait Sinwave_SaveService	Sauvegarde de la méta-progression et des règles
Rules	Sinwave_RunRules	Règles choisies pour la run, lues par les vagues, le joueur et la méta
HudModel	Sinwave_HudModel	Données affichées par l'interface

- **Remplacer un service** = enregistrer un autre objet sous la même clé. Les tests unitaires le font : ils montent un bus et des données de test autour d'un seul système.
- **Pourquoi pas une injection complète** : ZScript n'a ni constructeurs paramétrés, ni génériques, ni réflexion suffisante pour un conteneur. Passer un seul objet reste simple et explicite.
- **Limites assumées** : les dépendances d'un système ne se voient pas dans sa signature (chaque fichier les liste donc en tête), et une clé mal écrite n'est détectée qu'à l'exécution.

Le contenu dans des fichiers de données

Le code ne contient aucun ennemi, cercle, vertu, péché, article ou arène en dur : tout est décrit dans `src/data/`, au format `[type identifiant]` puis `clé = valeur`. C'est l'équivalent des ScriptableObject d'Unity : chaque bloc devient une définition, vérifiée au chargement (classe du moteur inexistante, ennemi inconnu, identifiant en double), avec le fichier et la ligne en cas d'erreur.

Fichier	Contenu	Lu par
systems.txt	Systèmes à créer, dans quel ordre, activés ou non	Sinwave_Game
arenas.txt	Arènes : carte, fichier de cercles, vie, vitesse, rythme, récompense	Sinwave_GameData

Fichier	Contenu	Lu par
waves/*.txt	Cercles d'une arène : vagues, durées, répits, malédiction, boss, tirage pondéré des ennemis	Sinwave_WaveSystem, Sinwave_CurseSystem
enemies.txt	Les 7 péchés, les boss et le reflet damné : classe du moteur, vie, vitesse, taille, XP, score, rage	Sinwave_WaveSystem, Sinwave_BossSystem
curses.txt	Les 7 malédictions : texte, classe de comportement, réglages	Sinwave_CurseSystem
upgrades.txt	8 vertus, 7 neutres, 7 péchés : famille, effets, corruption, maximum	Sinwave_UpgradeSystem
soul.txt	Balance de l'âme : bornes, butin, paliers, pente des choix, force du boss, épreuves	5 systèmes
shop.txt	Boutique : rayon, palier du Jugement demandé, prix, niveaux, effets	Sinwave_MetaSystem
judgement.txt	Jugement : bornes, neutralité, seuils des rayons, déplacement en fin de run	Sinwave_MetaSystem
difficulties.txt	Difficultés prédéfinies	Sinwave_RulesSystem
progression.txt	Courbe d'XP, composition des choix, gains d'indulgences, montée de la difficulté	plusieurs systèmes

Exemple, un péché dans `upgrades.txt` :

```
[upgrade sin_wrath]
name          = Colère
description   = +40 % de dégâts, mais tu subis +25 % de dégâts
kind          = sin
corruption    = 3
effects       = damage:0.4, vulnerability:0.25
max           = 2
```

- **Un seul langage d'effets** (`maxhealth` , `damage` , `speed` , `regen` , `armor` , `xpgain` , `give` , `aura` , `infiniteammo` ...) sert aux vertus, aux péchés, à la boutique, aux malédictions et aux paliers de l'âme, par un seul événement : `EffectGrantedEvent` .
- **Sans code**, on ajoute un ennemi, une vertu, un péché, un article, un cercle ou une arène complète (`tools/new-arena.ps1` , `guide docs/CREER-UNE-ARENE.md`).
- **Avec du code** seulement : un nouveau type d'effet, ou une nouvelle malédiction (une sous-classe de `Sinwave_Curse` , pattern Stratégie, choisie par son nom dans `curses.txt`).
- **Modding** : un fichier placé au même chemin dans une archive chargée après le jeu remplace l'original. Les tests s'en servent pour raccourcir les cercles.

Méta-progression : indulgences et Jugement

Ce qui reste d'une run à l'autre tient dans `Sinwave_MetaData` : indulgences, meilleur score, nombre de runs, Jugement de l'âme et niveau de chaque article acheté.

`Sinwave_MetaSystem` le gère sans connaître ni les cercles, ni le score, ni la corruption : il retient seulement ce que le bus annonce (`ScoreChanged` , `CircleEnded` , `CorruptionChanged`).

- **Gagner** : à la fin de la run, les indulgences viennent du score, des cercles franchis et d'un bonus de victoire qui dépend du verdict, le tout multiplié par la récompense de l'arène.
- **Juger** : le Jugement va de 0 (Grâce) à 100 (Corruption), neutre à 50. En fin de run, il bouge de 6, 10 ou 15 points selon le palier de l'âme atteint ; une run sans palier le ramène de 4 vers le neutre.
- **Dépenser** : la boutique a quatre rayons, ouverts selon le Jugement. Un article acheté une fois reste débloqué, même si son rayon se referme.
- **Appliquer** : au début de chaque run, chaque niveau acheté publie ses effets (`EffectGrantedEvent`), par le même chemin que les vertus.

Rayon	Esprit	Ouvret quand le Jugement est...
Armurerie	armes de départ	toujours
Grâce	le bien protège : vie, armure, défense, soins	≤ 40 (I), ≤ 25 (II), ≤ 10 (III)
Équilibre	objets compensés, un peu des deux	entre 35 et 65
Corruption	le mal frappe : dégâts, vitesse, XP	≥ 60 (I), ≥ 75 (II), ≥ 90 (III)

Une seule règle, un seul endroit : `Sinwave_JudgementDef` dit si un article est ouvert, combien de points il manque et où va le Jugement après une run. La méta-progression s'en sert pour refuser un achat, et l'interface pour griser un article et dessiner la jauge.

Sauvegarde : `Sinwave_CVarSaveService` écrit des CVars archivées (déclarées dans `CVARINFO`) et force l'écriture du `.ini` du moteur. Le système ne dépend que du contrat abstrait `Sinwave_SaveService` : un autre stockage se brancherait sans toucher à la logique.

L'interface séparée de la logique

L'interface ne modifie jamais le jeu directement : elle lit un modèle et envoie des commandes. ZScript le garantit à la compilation, avec deux portées : `play` (la simulation) et `ui` (l'affichage), où le code `ui` peut lire les objets `play` mais pas les modifier.

1. Le **présentateur** (`Sinwave_HudPresenter`) est un système comme les autres : il écoute le bus et traduit les événements en données d'affichage.
 2. Il les écrit dans le **modèle** (`Sinwave_HudModel`), un service.
 3. Le **HUD** et les **menus** (pause, choix de niveau, boutique, arènes, règles) ne font que lire ce modèle.
 4. Pour agir (choisir, acheter, reprendre), un menu envoie une **commande réseau** ; `Sinwave_Game` la traduit en événement sur le bus. C'est le seul chemin de l'interface vers le jeu.
- Les cinq menus héritent de `Sinwave_ChoiceMenu` : navigation, réglage gauche/droite, onglets, souris et dessin sont écrits une fois. Chaque menu ne décrit que ses options.
 - Clavier, souris et manette passent par les mêmes fonctions (`Move` , `AdjustSelected` , `Confirm` , `GoBack`).
 - `Sinwave_Canvas` place les textes sur un écran de 400 unités de haut, quel que soit le format de l'écran.

Désactiver ou remplacer un système

Un système se coupe avec `enabled = false` dans `data/systems.txt`, ou se remplace par une autre `class =`, sans toucher au code : aucun système ne référence un autre système, chacun ne connaît que des services et des événements. Les tests automatiques le prouvent en jouant des runs complètes sans l'interface : sans menu pour choisir, l'état Amélioration prend la première proposition au bout de 2 secondes.

```
[system trials]
class    = Sinwave_SoulTrialSystem
enabled = true
```

Système	Rôle	Le jeu sans lui
waves	Cercles, vagues, apparition des ennemis, boss	aucun ennemi
curses	Malédiction de chaque cercle	les cercles n'ont plus de règle spéciale
boss	Rage du boss et ses renforts	les boss n'entrent plus en rage
xp	Orbes d'expérience et montées de niveau	plus de montée de niveau
corruption	Balance de l'âme et ses paliers	l'âme reste à l'équilibre ; verdict toujours Purgatoire
loot	Soins et munitions selon l'âme	seul le butin d'origine de Doom reste
trials	Épreuves au bout de la balance	ni reflet damné, ni ange
threats	Alerte d'attaque dans l'angle mort	plus d'alerte
score	Score de la run	les indulgences ne viennent plus que des cercles
meta	Indulgences, Jugement, boutique, sauvegarde	rien n'est sauvegardé, la boutique ne vend plus rien
hud	Présentateur de l'interface	plus d'affichage ; la run se joue quand même
debug	Journal de tous les événements	plus de journal

La run minimale : le Vestibule

La run de 2 à 3 minutes demandée par le sujet est l'arène **Le Vestibule**, la première du choix d'arène. Un seul cercle (Paresse) : deux vagues d'une minute, puis le boss Charon. Les vagues finissent à 2 min 07 ; une partie automatique complète a duré 2 min 13, sur Freedoom comme sur Doom II.

Pour la lancer : démarrer le jeu, appuyer sur Utiliser (E), choisir « Le Vestibule », puis *Descendre*.

Ce que demande le sujet	Où on le voit dans le Vestibule
Une ou deux vagues d'ennemis	Deux vagues de plus en plus serrées, puis la vague du boss
Ramassage d'XP	Orbes bleues lâchées par les ennemis, attirées vers le joueur
Un choix d'amélioration	Menu vertu / neutre / péché à chaque niveau
Un score final	Écran de fin : score, victimes, niveau, temps, verdict
Sauvegarde de la méta-progression	Indulgences et Jugement écrits en fin de run, relus à la suivante

Rien dans le code ne distingue le Vestibule des autres arènes : il n'existe que par ses données (`arenas.txt` , `waves/vestibule.txt` , le boss `charon` dans `enemies.txt`) et sa carte (SW03). C'est la seule arène sous les 5 minutes de vagues ; le Purgatoire et les Limbes durent 6 à 8 minutes.

Tests et outils

Le jeu est vérifié par 77 tests unitaires et 4 scénarios de bout en bout, qui passent sur Freedoom et sur le vrai Doom II. GZDoom n'ayant pas de framework de test, tout est écrit dans le projet.

- **Tests unitaires** (`tests/smoke/zscript/sinwave_unittests.zs`) : bus, services, machine à états, lecteur de données, vagues, menaces, choix de niveau, balance de l'âme, Jugement, prix de la boutique, ordre de diffusion vu par l'interface. Chaque système testé est monté sur un bus de test, avec ses services, sans le reste du jeu.
- **Scénarios** (`tools/test.ps1`) : le script lance GZDoom avec une archive de test par-dessus le jeu (cercles raccourcis), joue par la console, puis vérifie dans le journal que chaque événement attendu arrive, dans l'ordre.

Scénario	Ce qu'il traverse
victoire	choix d'arène, cercle maudit, XP, montée de niveau, choix, pause, boss, verdict, sauvegarde puis relecture de la méta
mort	boutique refusée en run, mort du joueur, sauvegarde, boutique depuis l'écran de fin

Scénario	Ce qu'il traverse
boutique	achats réussis, refusés ou verrouillés par le Jugement, défi personnalisé, voyage vers une autre arène, achats appliqués
interface	aucun menu bloqué après un changement de carte, boutique depuis l'écran de fin

Outil (<code>tools/</code>)	Rôle
<code>check.ps1</code>	Construit l'archive et vérifie que le ZScript compile (Ctrl+Maj+B dans VS Code)
<code>test.ps1</code>	Tests de bout en bout ; <code>-Iwad doom2</code> pour le vrai Doom II
<code>run.ps1</code> , <code>play.ps1</code>	Construit puis lance le jeu
<code>package.ps1</code>	Build Windows à rendre : <code>dist/Sinwave-win64.zip</code> , avec un lanceur Freedoom et un lanceur Doom II
<code>generate-arena.ps1</code>	Génère les cartes (entonnoir, château, vestibule)
<code>new-arena.ps1</code>	Crée une arène personnalisée, sans code

Le build Windows contient GZDoom, `sinwave.pk3` et Freedoom, seul IWAD redistribuable ; Doom II, jeu commercial, n'est jamais copié. Sur GitHub, chaque push construit l'archive, et chaque tag `v*` publie le build dans *Releases*.

Limites connues et choix assumés

La principale limite est l'ordre de diffusion du bus, synchrone : il dépend de l'ordre des systèmes dans `systems.txt` , et aucun système ne doit compter dessus.

Limite	Conséquence	Ce qui est fait aujourd'hui
Bus synchrone, ordre fixé par <code>systems.txt</code>	Un événement publié pendant la diffusion d'un autre arrive avant lui chez les abonnés suivants	Les systèmes retiennent la dernière valeur annoncée ; deux bugs de ce type corrigés (malédiction invisible, âme affichée à 0/30) ; un test unitaire rejoue cet ordre

Limite	Conséquence	Ce qui est fait aujourd'hui
Dépendances cachées dans le Service Locator	Elles ne se voient pas dans la signature ; une clé mal écrite n'apparaît qu'à l'exécution	Chaque fichier de système liste en tête ce qu'il écoute, publie et utilise
Données vérifiées au lancement seulement	Une faute de frappe n'est signalée qu'au chargement de la carte	Message dans la console, avec le fichier et la ligne
Un événement alloué à chaque publication	Des objets à collecter	Sans effet mesurable à cette échelle
Certaines malédictions parcourent tous les monstres tous les 4 à 5 tics	Coûteux pour de très grosses vagues	Sans problème pour quelques dizaines d'ennemis
Sauvegarde dans un <code>.ini</code> lisible	Un joueur peut changer ses indulgences	Acceptable pour un prototype solo
Une carte par arène	L'arène se retrouve d'après la carte chargée	Chaque arène a sa carte
Arènes non cumulées entre archives	Un mod qui ajoute une arène remplace tout <code>arenas.txt</code>	À faire (voir plus bas)
IA de Doom sans recherche de chemin	Un ennemi peut apparaître derrière une cloison	Cartes ouvertes ; <code>spawn = points</code> pour une carte cloisonnée
Tests qui ouvrent une fenêtre GZDoom	Pas de tests sur le serveur d'intégration continue	GitHub construit seulement l'archive et le build
Jeu solo	Le code suppose un seul joueur	Assumé pour ce prototype

Avec plus de temps

La première chose à faire serait de ne plus dépendre de l'ordre de diffusion, la limite qui a déjà causé deux bugs.

1. Abonnements au bus avec une priorité explicite, pour ne plus dépendre de l'ordre de `systems.txt`.
2. Un outil qui valide les fichiers de données hors du jeu, lancé avant chaque commit.
3. Le cumul des arènes entre archives, pour que les mods ajoutent une arène sans rien remplacer.

4. Des défis à modificateurs spéciaux (pas de soin, ennemis explosifs) en plus des réglages chiffrés.
5. Des sprites, des sons et des musiques propres à chaque péché, et d'autres boss.
6. La traduction des textes par le fichier `LANGUAGE` du moteur, au lieu de chaînes en français dans le code.